

Amrita Vishwa Vidyapeetham

Amrita School of Computing, Amritapuri

BTech Computer Science and Engineering (AI)

22AIE311 – Software Engineering (Project based)

LAB SHEET 2 — ELABORATION PHASE

AI-Powered Personal Resource Optimization & Subscription Intelligence System (PROSIT)

PART A – USE CASE MODELING [CO04]

Task 1: Identify Actors

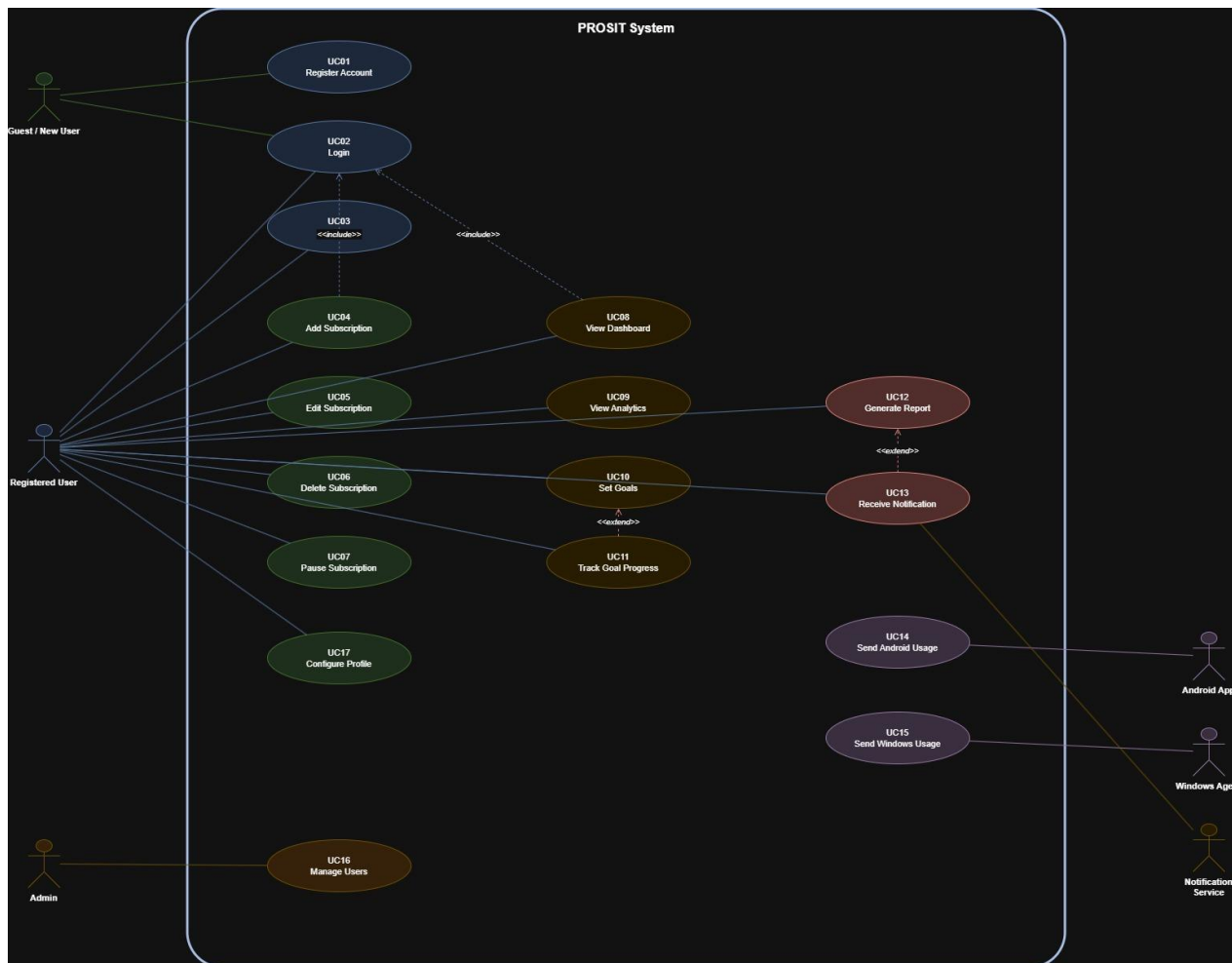
Sl. No	Actor Name	Description
1	Registered User	Primary actor. An authenticated user who manages subscriptions, monitors device usage, views analytics, sets goals, and receives notifications.
2	Guest / New User	Unauthenticated visitor who can register for a new account or view the landing page.
3	Admin	System administrator who manages user accounts, monitors system health, and configures platform-level settings.
4	Android App	Companion mobile application running on the user's Android device. Tracks foreground app usage and sends data to the backend API.
5	Windows Agent	Desktop monitoring agent running on the user's Windows machine. Tracks active application/process usage and pushes data to the backend.
6	Notification Service	External service (Firebase Cloud Messaging / SendGrid) that delivers push notifications and email alerts to users.

Task 2: Identify Use Cases

Use Case ID	Use Case Name	Actor(s)	Description
UC01	Register Account	Guest User	New user registers with name, email, and password. System validates and creates account.
UC02	Login	Registered User, Guest User	User authenticates via email/password; system issues JWT token upon success.
UC03	Logout	Registered User	User invalidates session/token and is redirected to landing page.
UC04	Add Subscription	Registered User	User adds a new subscription with name, cost, billing cycle, and category.
UC05	Edit Subscription	Registered User	User modifies details of an existing subscription.
UC06	Delete Subscription	Registered User	User permanently removes a subscription from the system.
UC07	Pause Subscription	Registered User	User temporarily pauses a subscription; cost is excluded from active totals.
UC08	View Dashboard	Registered User	User views aggregated spending, usage statistics, and AI-generated insights on main dashboard.

UC09	View Analytics	Registered User	User explores detailed metrics: cost-per-hour, ROI, utilization rate, opportunity cost.
UC10	Set Goals	Registered User	User defines personal time or money goals (e.g., reduce streaming spend to \$20/month).
UC11	Track Goal Progress	Registered User	User monitors how current behavior compares against defined goals.
UC12	Generate Report	Registered User	System generates a weekly/monthly summary report of subscription spend and usage.
UC13	Receive Notification	Registered User, Notification Svc	User receives email/push alerts for renewals, low usage warnings, or recommendations.
UC14	Send Android Usage Data	Android App	Android companion app collects foreground app session data and pushes it to backend API.
UC15	Send Windows Usage Data	Windows Agent	Windows agent collects active process/window usage and posts it to backend API.
UC16	Manage Users	Admin	Admin views, deactivates, or removes user accounts from the admin panel.
UC17	Configure Profile	Registered User	User updates display name, password, notification preferences, and connected devices.

Task 3: Use Case Diagram



The Use Case Diagram captures all actors and their interactions with the PROSIT system. Key relationships include:

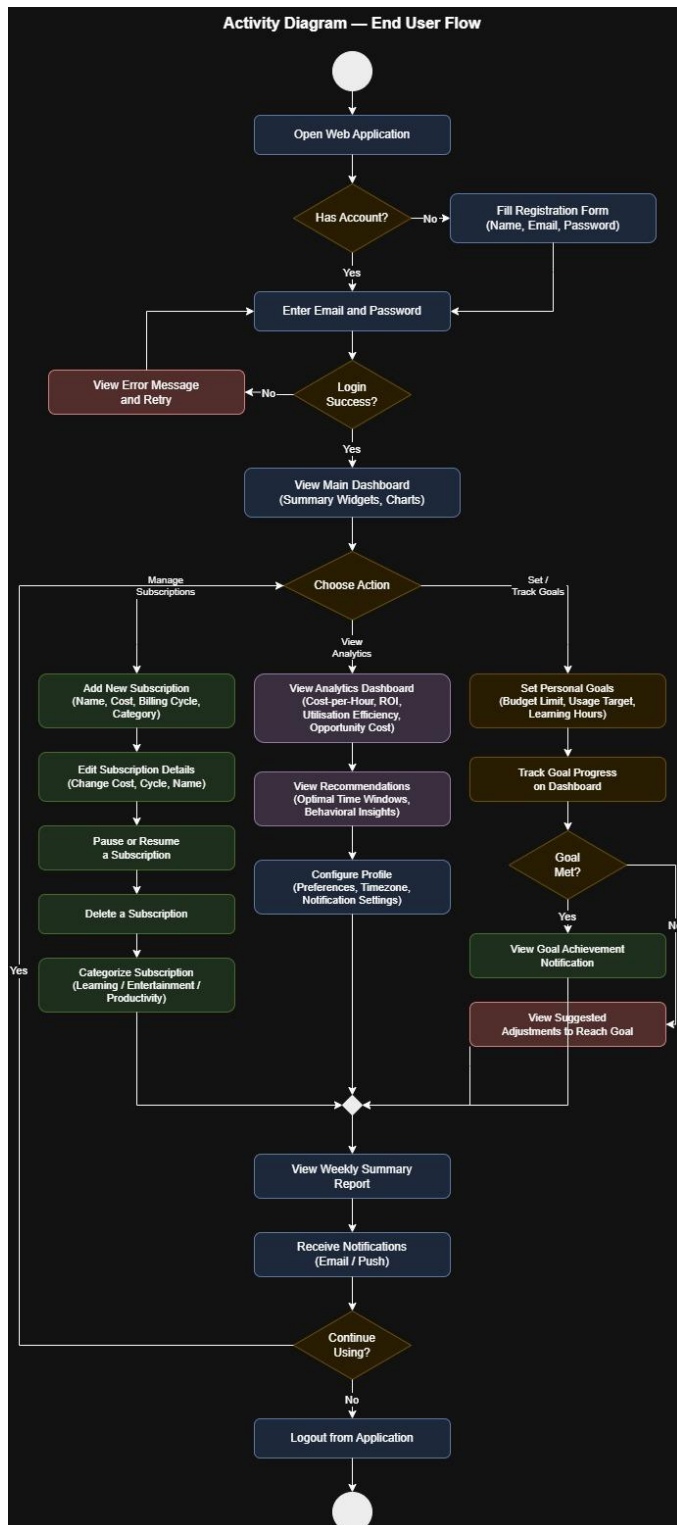
- «include»: Login is included by Add/Edit/Delete Subscription, View Dashboard, View Analytics, Set Goals.
- «extend»: Receive Notification extends Generate Report (optional alert on completion).
- «extend»: Track Goal Progress extends Set Goals.
- Android App and Windows Agent interact only with UC14 and UC15 respectively.

PART B – ACTIVITY MODELING [CO04]

Task 4: Key Activities in System Workflow

#	Activity	Triggered By
1	User Registration & Email Verification	Guest User
2	User Login / Authentication (JWT issue)	Registered User
3	Add / Edit / Delete / Pause Subscription	Registered User
4	Collect Android App Usage Data	Android App (background service)
5	Collect Windows Process Usage Data	Windows Agent (background process)
6	Aggregate & Store Usage Records	Backend API
7	Compute Cost-per-Hour & ROI Metrics	Analytics Engine
8	Generate AI-Powered Recommendations	Analytics Engine
9	Render Dashboard with Charts	Frontend (React)
10	Set / Update Goals	Registered User
11	Evaluate Goal Progress	Analytics Engine
12	Generate Weekly / Monthly Report	Scheduler (cron job)
13	Send Notification (email / push)	Notification Service
14	Admin: Manage User Accounts	Admin

Task 5: Activity Diagram

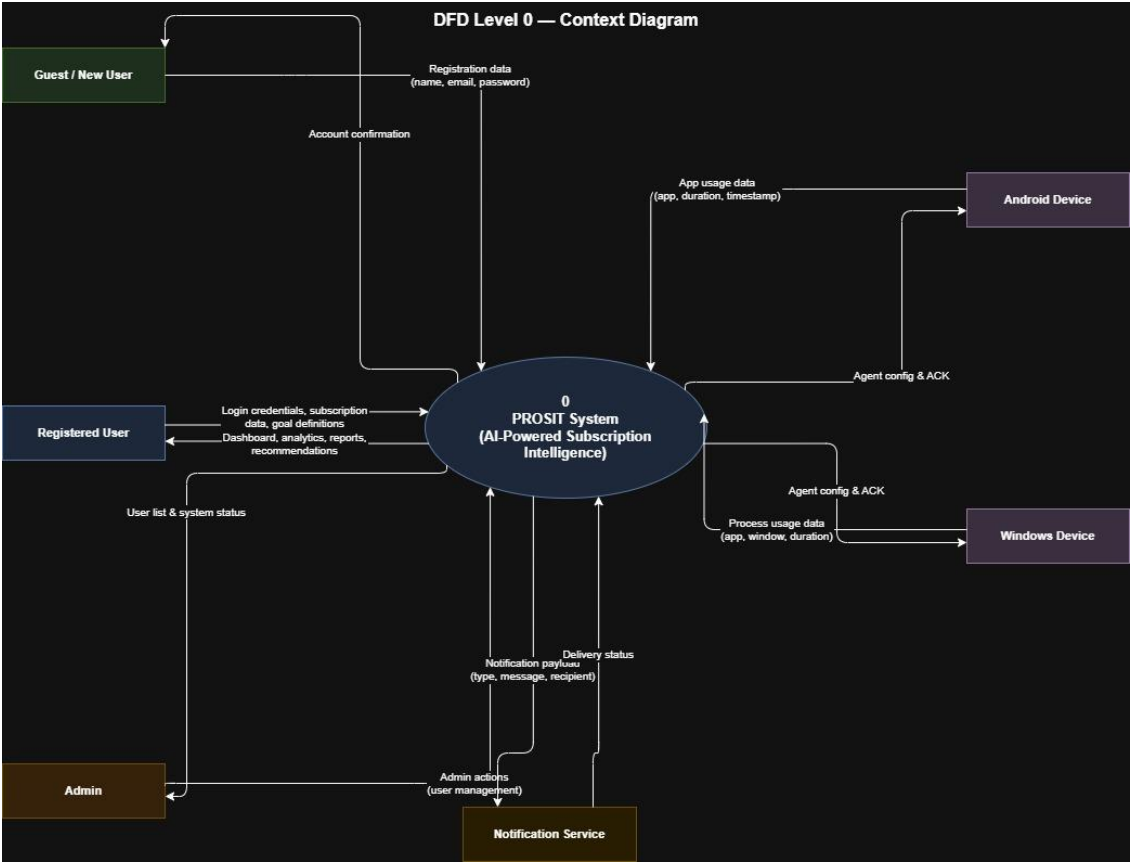


The Activity Diagram models the primary user workflow: from login through subscription management to analytics viewing and goal tracking. Decision points include authentication success/failure, device monitoring consent, and goal achievement status.

Task 6: System Inputs and Outputs

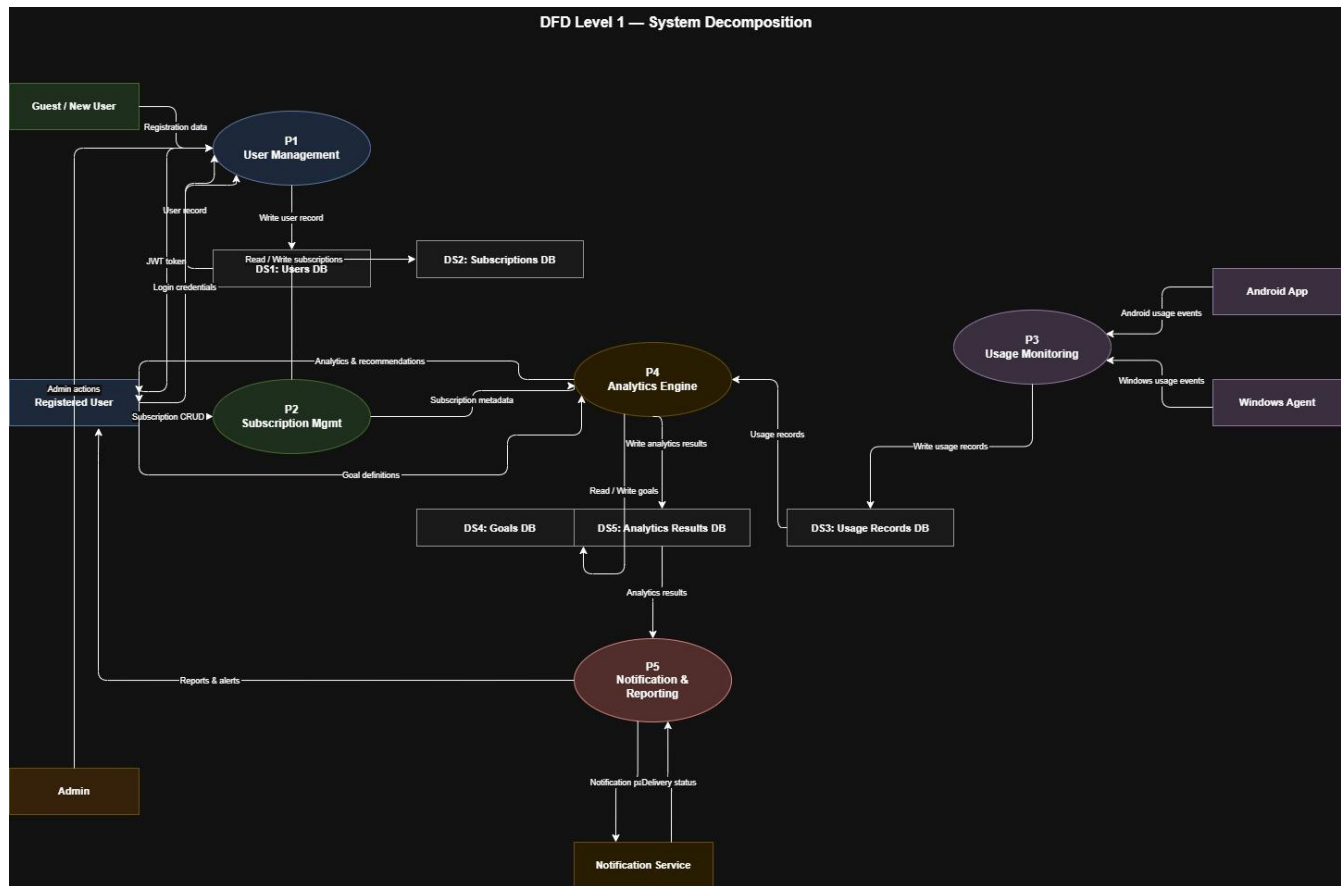
Inputs	Outputs
User credentials (email, password)	JWT access token / authentication response
Subscription details (name, cost, cycle, category)	Subscription list with analytics metadata
Android app usage data (app name, duration, timestamp)	Analytics report (cost-per-hour, ROI, utilization)
Windows process usage data (process name, window title, duration)	AI recommendations & optimization suggestions
Goal definitions (target, metric, deadline)	Goal progress report
User profile updates	Weekly / monthly summary notifications
Report generation request	PDF / JSON usage & cost report
Admin actions (deactivate/remove user)	Updated user status confirmation

Task 7: DFD Level 0 — Context Diagram



The context diagram represents the entire PROSIT system as a single process. External entities: Registered User, Guest User, Admin, Android Device, Windows Device, and Notification Service. All data flows to/from the system are shown at this level.

Task 8: DFD Level 1



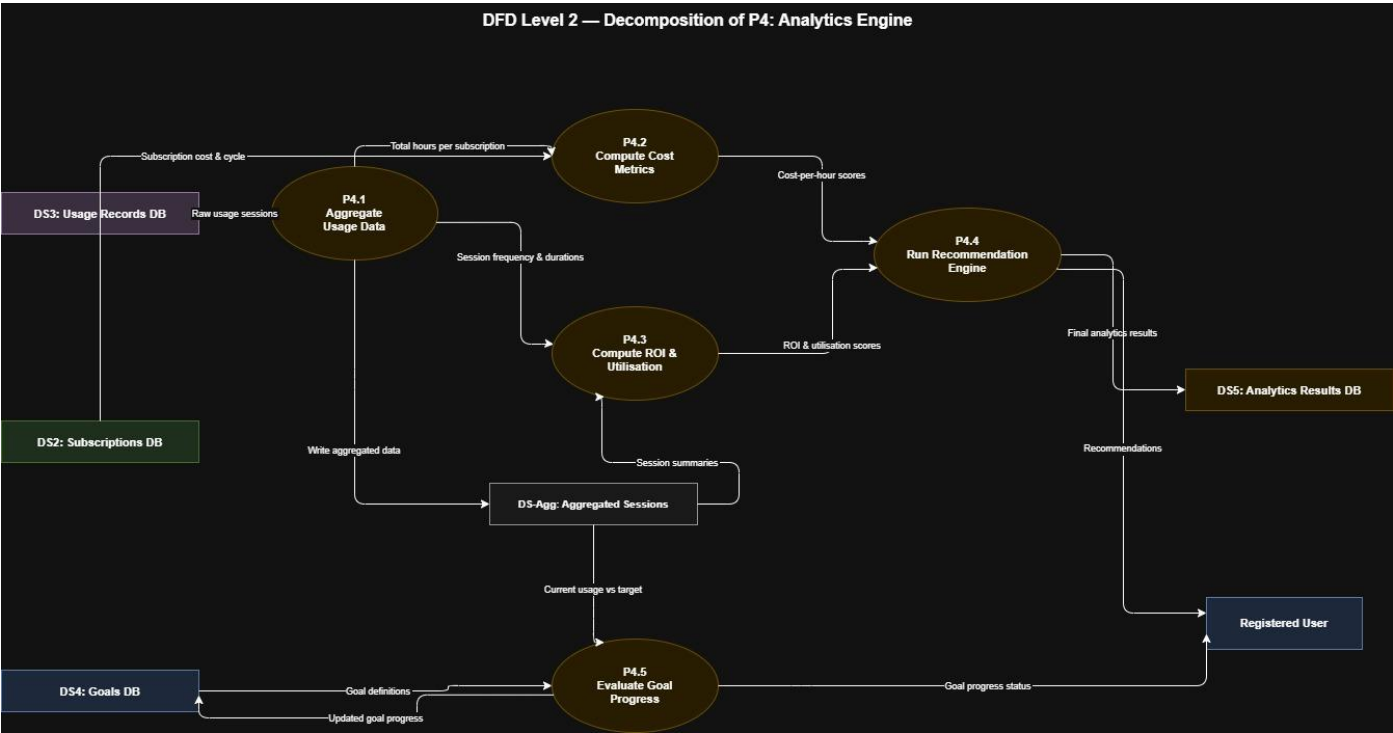
Level 1 decomposes the system into five major sub-processes:

P1 – User Management | P2 – Subscription Management | P3 – Usage Monitoring

P4 – Analytics Engine | P5 – Notification & Reporting

Data stores: DS1 Users DB, DS2 Subscriptions DB, DS3 Usage Records DB, DS4 Goals DB.

Task 9: DFD Level 2



Level 2 further decomposes P4 – Analytics Engine into:

- P4.1 – Aggregate Usage Data
- P4.2 – Compute Cost Metrics
- P4.3 – Generate ROI & Utilization
- P4.4 – Run Recommendation Engine
- P4.5 – Evaluate Goals

PART D – CLASS DIAGRAM MODELING [CO04]

Task 10: Identify Classes

Sl. No	Class Name	Type	Description
1	User	Entity	Stores registered user credentials, profile info, and preferences.
2	Subscription	Entity	Represents a subscription service with cost, billing cycle, and category.
3	UsageRecord	Entity	Captures app/process usage session: device type, app name, duration, timestamp.
4	Goal	Entity	Stores user-defined time or money goals with target metrics and deadlines.
5	AnalyticsResult	Entity	Stores computed metrics: cost-per-hour, ROI, utilization rate for a subscription.
6	Notification	Entity	Stores notification payload, type, status, and recipient for audit.
7	Report	Entity	Represents a generated weekly/monthly usage and spend report.
8	LoginForm	Boundary	Frontend UI component for user login (email + password form).
9	DashboardView	Boundary	Frontend component rendering the main analytics dashboard with charts.

10	SubscriptionFormView	Boundary	Frontend form UI for adding or editing a subscription.
11	GoalFormView	Boundary	Frontend UI for creating or updating a personal goal.
12	DeviceAgentInterface	Boundary	API interface through which Android/Windows agents push usage data.
13	AuthController	Control	Handles registration, login, JWT issuance, and session validation.
14	SubscriptionController	Control	Orchestrates CRUD operations on subscriptions and category management.
15	UsageController	Control	Receives, validates, and stores device usage data from agents.
16	AnalyticsController	Control	Triggers analytics computation, retrieves results, and surfaces recommendations.
17	GoalController	Control	Manages goal creation, updates, and progress evaluation logic.
18	NotificationController	Control	Triggers notification dispatch via external service; manages delivery status.
19	ReportController	Control	Schedules and generates periodic usage/cost summary reports.

Task 11: Attributes and Methods

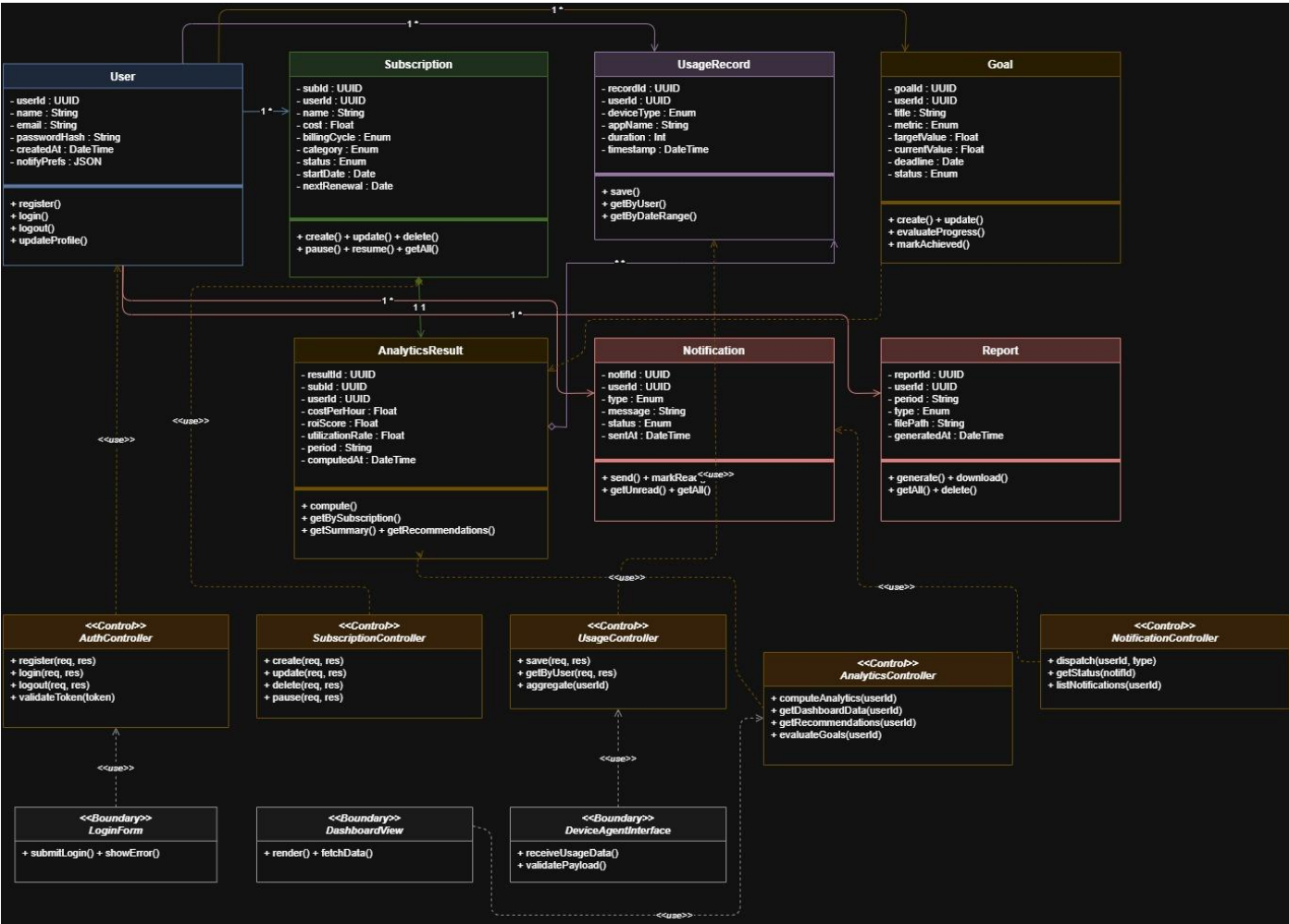
Class Name	Attributes	Methods
User	userId: UUID name: String email: String passwordHash: String createdAt: DateTime notifyPrefs: JSON	register() login() logout() updateProfile() deleteAccount()
Subscription	subId: UUID userId: UUID name: String cost: Float billingCycle: Enum category: Enum status: Enum startDate: Date nextRenewal: Date	create() update() delete() pause() resume() getAll()
UsageRecord	recordId: UUID userId: UUID deviceType: Enum appName: String duration: Int (seconds) timestamp: DateTime	save() getByUser() getBySubscription() getByDateRange()
Goal	goalId: UUID userId: UUID title: String metric: Enum targetValue: Float currentValue: Float deadline: Date status: Enum	create() update() evaluateProgress() markAchieved()
AnalyticsResult	resultId: UUID subId: UUID userId: UUID costPerHour: Float roiScore: Float utilizationRate: Float period: String computedAt: DateTime	compute() getBySubscription() getSummary() getRecommendations()
Notification	notifId: UUID userId: UUID type: Enum message: String status: Enum sentAt: DateTime	send() markRead() getUnread() getAll()
Report	reportId: UUID userId: UUID period: String type: Enum filePath: String generatedAt: DateTime	generate() download() getAll() delete()
AuthController	(no state — controller)	register(req, res) login(req, res) logout(req, res) validateToken(token) refreshToken(req, res)

AnalyticsController	(no state — controller)	computeAnalytics(userId) getDashboardData(userId) getRecommendations(userId) evaluateGoals(userId)
---------------------	-------------------------	---

Task 12: Identify Relationships

Class 1	Relationship	Class 2	Description
User	Association (1..*)	Subscription	A user owns one or more subscriptions.
User	Association (1..*)	UsageRecord	A user has many usage records across devices.
User	Association (1..*)	Goal	A user sets one or more goals.
User	Association (1..*)	Notification	A user receives multiple notifications.
User	Association (1..*)	Report	A user can generate multiple reports.
Subscription	Association (1..*)	UsageRecord	A subscription is associated with multiple usage sessions.
Subscription	Composition (1..1)	AnalyticsResult	Each subscription has exactly one computed analytics result (recomputed periodically).
AnalyticsResult	Aggregation (1..*)	UsageRecord	Analytics result aggregates multiple usage records.
Goal	Association (0..*)	AnalyticsResult	Goals are evaluated against computed analytics results.
AuthController	Dependency	User	AuthController reads and creates User entities.
SubscriptionController	Dependency	Subscription	SubscriptionController manages Subscription entities.
AnalyticsController	Dependency	AnalyticsResult	AnalyticsController triggers and reads AnalyticsResult.
GoalController	Dependency	Goal	GoalController manages Goal entities and evaluates progress.
NotificationController	Dependency	Notification	NotificationController creates and dispatches Notification.
ReportController	Dependency	Report	ReportController generates and stores Report entities.
LoginForm	Dependency	AuthController	LoginForm invokes AuthController for authentication.
DashboardView	Dependency	AnalyticsController	DashboardView fetches analytics data via AnalyticsController.
SubscriptionFormView	Dependency	SubscriptionController	SubscriptionFormView calls SubscriptionController for CRUD.
DeviceAgentInterface	Dependency	UsageController	Device agents push data through UsageController.

Task 13: Class Diagram



The class diagram includes all 19 classes across Entity, Boundary, and Control layers, showing full attributes, methods, multiplicities, and relationship types (association, composition, aggregation, dependency).

Task 14: Justification of Design

The class model follows a three-layer MVC-aligned architecture. Entity classes (User, Subscription, UsageRecord, Goal, AnalyticsResult, Notification, Report) directly map to database tables, ensuring a clean ORM boundary. Boundary classes represent the frontend UI components and external API interfaces, deliberately separating presentation concerns from business logic. Control classes act as service/controller layers handling orchestration — each controller is scoped to a single domain (auth, subscriptions, analytics, etc.) to enforce the Single Responsibility Principle. Relationships were identified from data ownership (User owns Subscription via foreign key) and computation dependencies (AnalyticsResult aggregates UsageRecords). Composition was used for AnalyticsResult-Subscription because the result has no meaning outside its parent subscription. This design supports scalability, testability, and clean API boundary definition.